

Visualization Library Documentation

Python offers a rich ecosystem of data visualization libraries, each suited for different needs ranging from basic static plots to complex interactive dashboards and specialized visualizations like maps.

The most popular and widely used libraries include Matplotlib, Seaborn, Plotly, Bokeh, Altair,

MATPLOTLIB

Matplotlib is a powerful Python library used for **data visualization**. It helps create basic, statistical and interactive charts. Its code length is longer compare to other library but, simple and easy to understand.

The most common types of simple graphs include line plots, scatter plots, bar charts, and histograms.

Component of the plot:

- Figure → Entire window full canvas or whole chart window. It is the outer container that holds everything
- Axes → Graph area actual plotting area where data is drawn
- Title → Heading of the graph, it explains what the chart is about
- Labels → Names of X-axis and Y-axis, they explain what data is shown.
- Legend → Tells which line/bar represents what, used when multiple data are shown.

Initially we have to install matplotlib using below given. Its one-time step.

```
pip install matplotlib
```

Import matplotlib library using below mentioned code. Also import numpy for mathematical operation

Code to import [matplotlib](#) :

```
import matplotlib.pyplot as plt
```

Code to import [numpy](#) :

```
import numpy as np
```

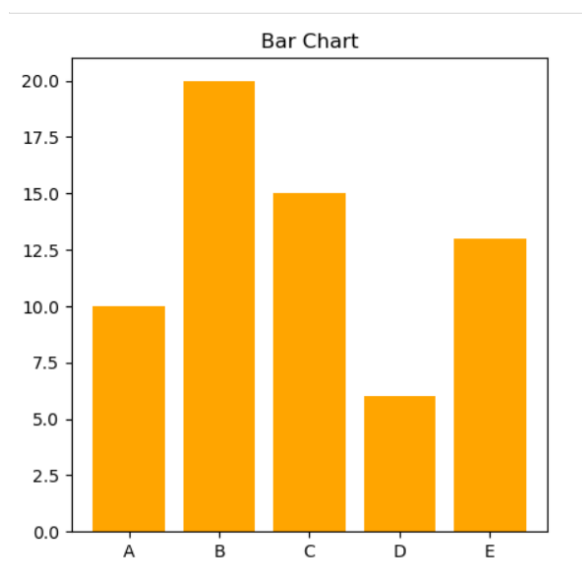
BAR CHART:

A bar chart is useful for comparing categorical data. Each categorical value claims one bar, and the length of each bar corresponds to the bar's value. The taller or longer the bar, the higher value it represents.

Code:

```
## BAR CHART
categories = ['A','B','C','D','E']
values = [10,20,15,6,13]
plt.figure(figsize=(5,5))          # width=5, height=5
plt.bar(categories, values, color='orange')
plt.title("Bar Chart")
plt.show()
```

Output:



`bar()` function takes arguments that describes the layout of the bars. `plt.bar()` is used along with the parameter x, y, bar colour and width ect. (x=categories, y=values)

To get horizontal bar chart change `plt.bar` to `plt.barh()`

LINE CHART:

A line plot is ideal for visualizing trends over a continuous range, such as time.

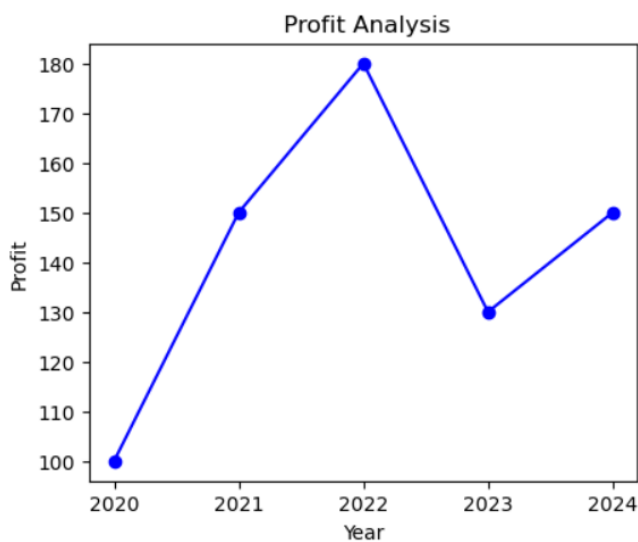
Code:

```
## LINE CHART
x = [2020, 2021, 2022, 2023, 2024] # years
y = [100, 150, 180, 130, 150]      # profit
plt.figure(figsize=(5,4))
plt.plot(x, y, color='blue', marker='o')
plt.title(" Profit Analysis")
plt.xlabel("Year")
plt.ylabel("Profit")

plt.xticks(x)                        # Show only year values on x-axis

plt.show()
```

Output:



By default, the `plot()` function draws a line from point to point. `plt.xticks()` function in matplotlib is used to get or set the properties of the x-axis ticks and their corresponding labels

SCATTERED PLOT:

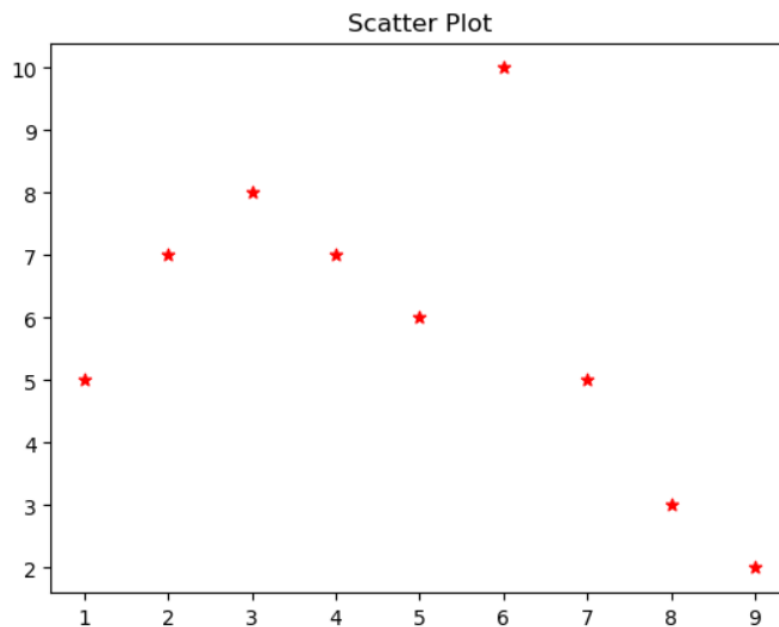
A scatter plot is used to observe relationships between two different variables by displaying individual data points as markers.

Code:

```
x = [1,2,3,4,5,6,7,8,9]
y = [5,7,8,7,6,10,5,3,2]

plt.scatter(x,y, color='red', marker='*')
plt.title("Scatter Plot")
plt.show()
```

Output:



`plt.scatter()` function offers several parameters for customization. Parameters like colour and marker to choose the colour for the data point and marker style correspondingly.

HISTOGRAM:

A histogram is a chart that plots the distribution of a numeric variable's values as a series of bars. Each bar typically covers a range of numeric values called a bin.

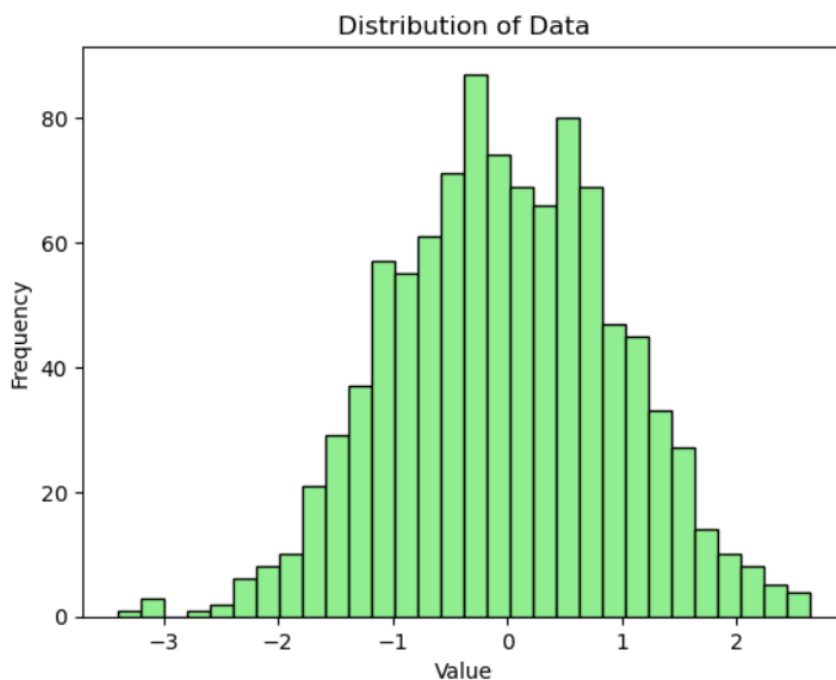
A bar's height indicates the frequency of data points with a value within the corresponding bin.

Code:

```
data = np.random.randn(1000) # 1000 data points from a normal distribution

plt.hist(data, bins=30, color='lightgreen', edgecolor='black')
plt.title("Distribution of Data")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

Output:



`plt.hist()` function automatically groups your data into bins and displays the frequency of values in each. `bins` controls the number of bars and you can pass an integer or a list.

SEABORN

Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.

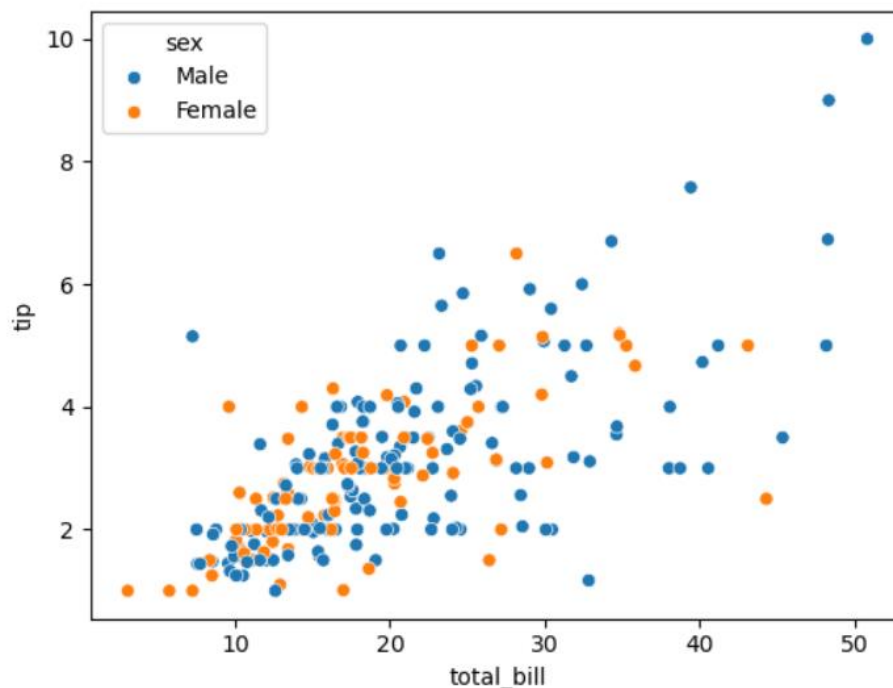
It offers various types of plots, including scatter plots, line plots, bar plots, histograms, box plots, violin plots, and heatmaps, among others. It is a powerful but easy-to-use data visualization tool.

SCATTER PLOT:

Code:

```
sns.scatterplot(x="total_bill", y="tip", hue="sex", data=tips)
plt.show()
```

Output:



`sns.scatterplot()` function is used to visualize the relationship between two numerical variables, where each data point is represented as a dot.

`hue` parameter is used to colour the data points based on a third variable, allowing you to visualize an additional dimension of information within the same plot.

`data` parameter is the input panda data frame.

BOX PLOT:

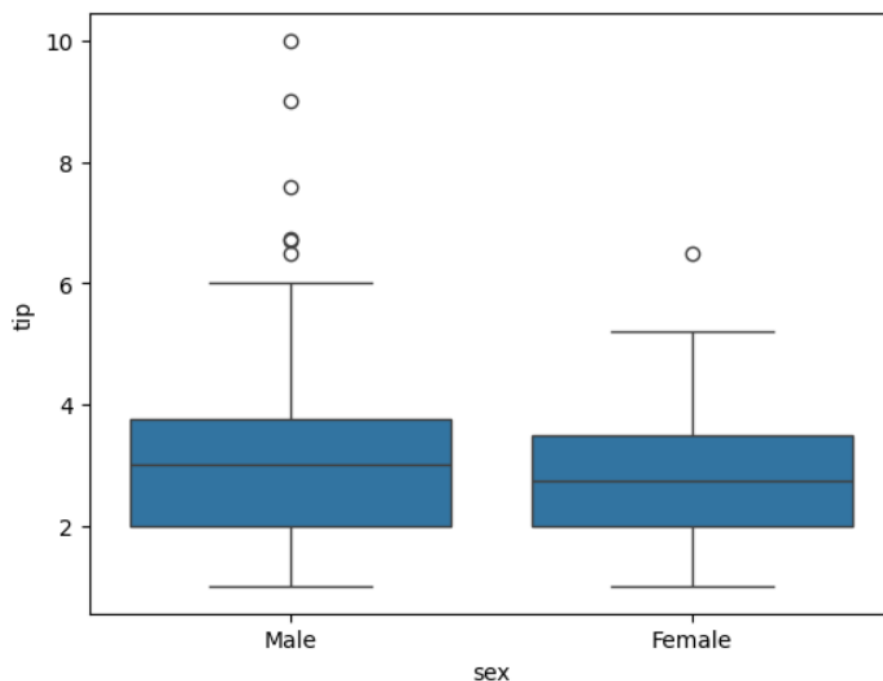
A box plot or box-and-whisker plot shows the distribution of quantitative data in a way that facilitates comparisons between variables or across levels of a categorical variable.

The spread and size of the box reveal the concentration and variability of the data. A longer box suggests greater variability, while the whiskers give additional context about the distribution range.

Code:

```
sns.boxplot(x="sex", y="tip", data=tips)
plt.show()
```

Output:



`sns.boxplot()` function is used to create a box-and-whisker plot, which visualizes the distribution of quantitative data across different categories.

Box: Represents the interquartile range (IQR), spanning from the first quartile (Q1, 25th percentile) to the third quartile (Q3, 75th percentile)

COUNT PLOT:

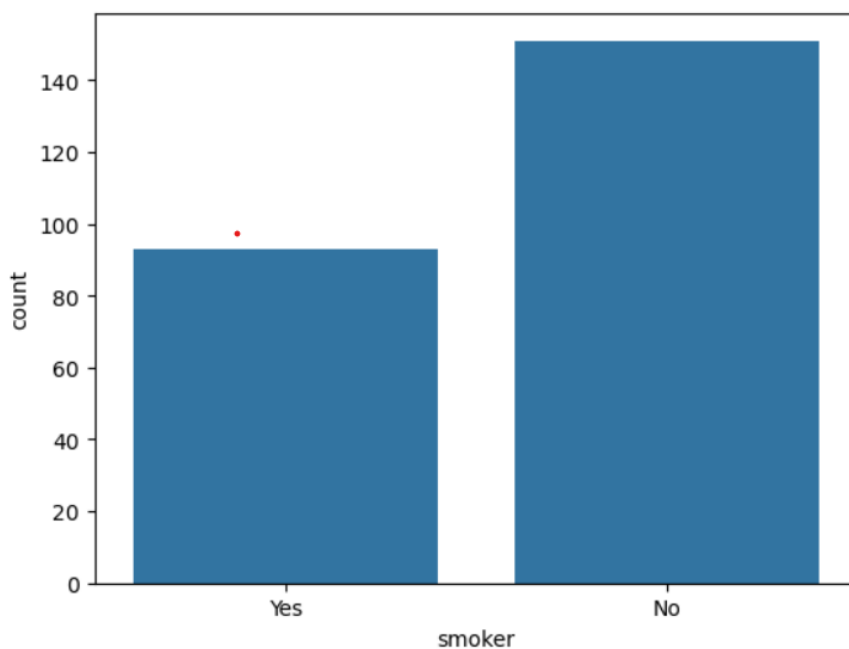
It displays the count of observations in each categorical bin as a bar plot. Allow automatic counting, which means does not require to pre-calculate the counts, it does the aggregation internally.

No need to manually count the categories and then created bar charts.

Code:

```
sns.countplot(x="smoker", data=tips)
plt.show()
```

Output:



`countplot()` function takes input data in many forms, such as wide-form data, long-form data, arrays or a list of vectors.

x parameters take names of variables as input that plot the long form data.

HEAT MAP:

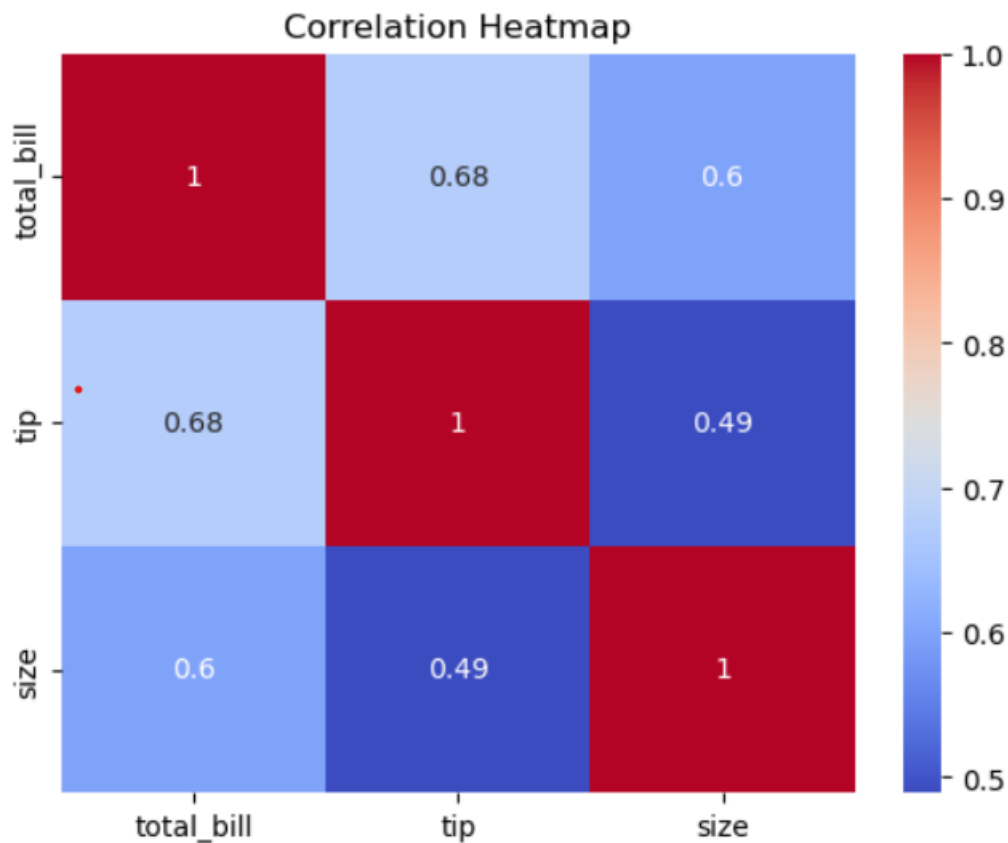
Heatmap is a plot of rectangular data as a color-encoded matrix. It is a powerful tool in Python for visualizing patterns, correlations, and anomalies in a dataset, commonly used in exploratory data analysis and machine learning.

Code:

```
corr = tips.corr(numeric_only=True)

sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

Output:



It initially compute the correlation matrix of your data using pandas `.corr()` function.

Heat map is plotted using the `sns.heatmap()` function, which offers extensive customization options. Parameter `annot` if True, displays numerical values inside the cells. `cmap` is colormap for mapping data values to colours.

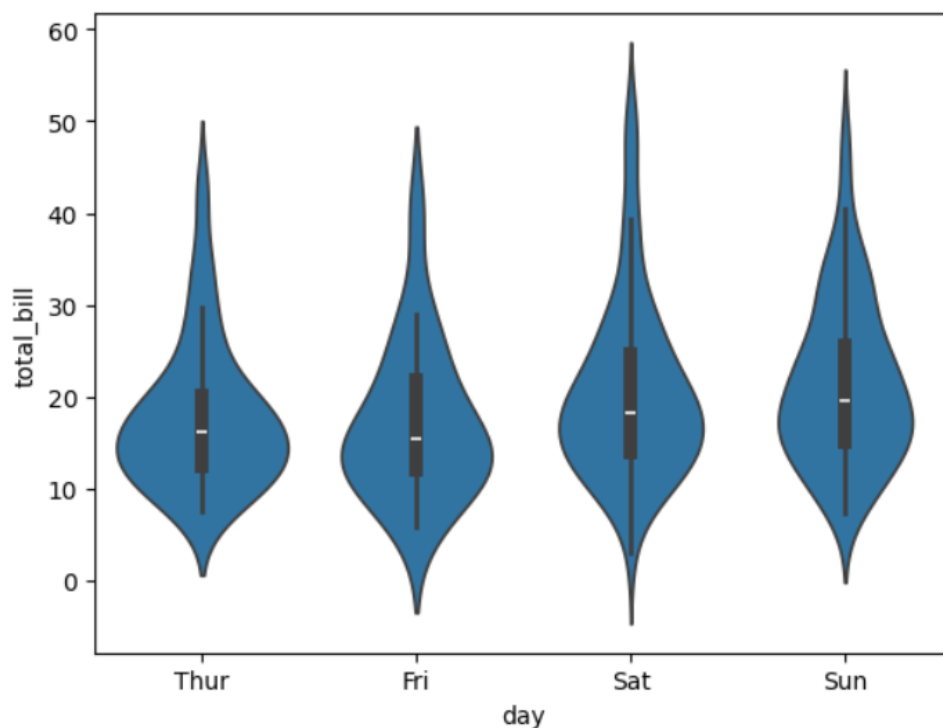
VIOLIN PLOT :

Violin plots are used when you want to observe the distribution of numeric data, especially useful when you want to make a comparison of distributions between multiple groups

Code:

```
sns.violinplot(x="day", y="total_bill", data=tips)
plt.show()
```

Output:



Violin plot is created using the `sns.violinplot()` function.

Outer Shape represents the KDE (kernel density estimation), White Dot is the median value of the data.

Thin Inner Line is the interquartile range (IQR), Thicker Inner Line is the whiskers extending to show the typical range of the data, excluding outliers.

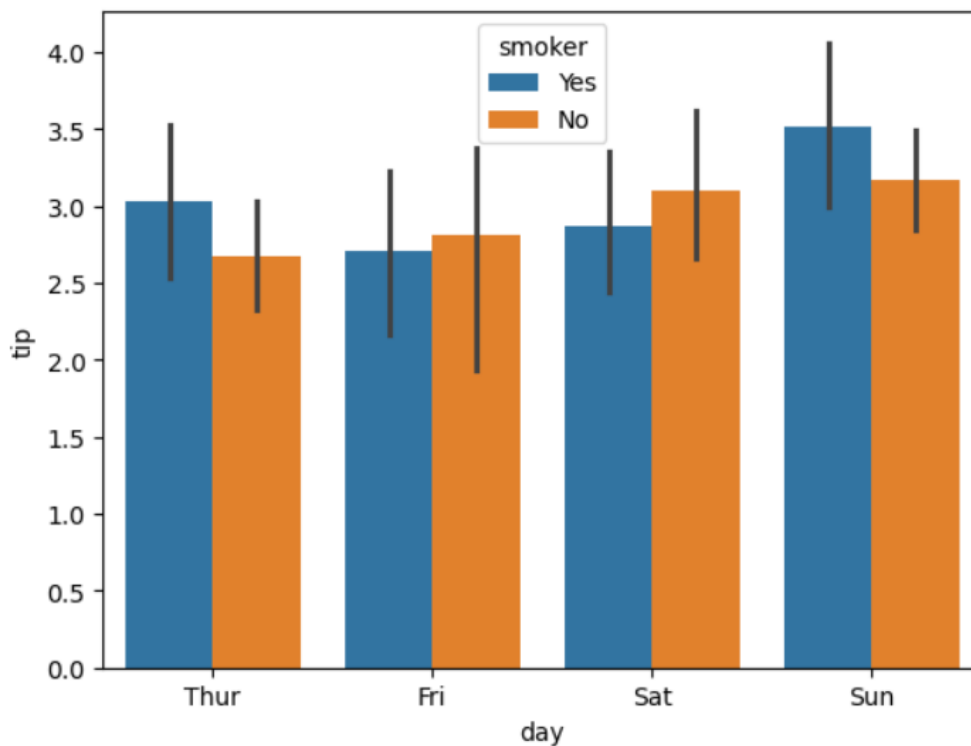
BAR PLOT:

Bar plot that visualizes the relationship between a categorical variable and a numeric variable. Each categorical value claims one bar, and the length of each bar corresponds to the bar's value.

Code:

```
sns.barplot(x="day", y="tip", hue="smoker", data=tips)
plt.show()
```

Output:



`sns.barplot` function takes in the parameters data , x and y . It then plots a bar plot using data as the dataset for the plot.